Generics In JAVA

DAY 62
10/09/2025

General ⟶ The word generics is derived from the word General

The Opposite Meaning to the word general is Specific

General ⟶ Specific

Specific means using Some particular tool or technology

Generic using Same tool for Multiple purpose

Generic ⟶ using Single tool for Multiple purpose

Specific ⟶ using Some tool for Specific purpose

What are generics in JAVA ?

Generics allow you to write classes, interfaces, and methods with type parameters.

This Means Instead of Specifying a fixed data type, you can write code that Works with different data types in a type-Safe Way.

Why we Need Generics ?

1. Type Safety
   Prevents runtime ClassCastException by checking types at compile-time
   Bcz compile time Errors are Easy to Handle Compared to Runtime Errors.

2. Code Reusability
Write a Single generic class/method that Works With different data types.

3. cleaner Code
No Need for Explicit Casting

✅ **Summary:**

- Generics = Compile-time type safety + Reusability + Cleaner Code.
- Used in Collections, Streams, and custom reusable classes/methods.
- Key features: **Type Parameters, Bounded Types, Wildcards.**

🔷 **Analogy 1: Lunch Box** 🍱

Imagine a **lunch box**.

- A **normal lunch box (without generics)**:
  You can put anything inside—rice, books, mobile phone. But when you open it, you don't know what's inside until runtime. If you expect rice but find a phone, you'll be shocked (runtime error 😅).
- A **typed lunch box (with generics)**:
  If you say this lunch box is **Box<Rice>**, only rice can go inside.
  If you try to put a phone, the shopkeeper (compiler) stops you right there.
  👉 This ensures safety and avoids surprises later.

🔷 **Analogy 2: Shopping Bags** 🛍️

Think of **supermarket shopping bags**.

- A **non-generic bag**: One bag for all items—milk, chips, shampoo. You need to check carefully before using items (casting).
- A **generic bag**: Bag labeled **"Fruits only"** or **"Vegetables only"**.
  - A **FruitBag** → only apples inside.
  - A **FruitBag** → only mangoes inside.

  No need to check at home; you already know what's inside.

🔷 **Analogy 3: Coffee Cup** ☕

A coffee shop gives you cups:

- **Raw Cup (without generics)**: They give you a cup, but you don't know if it contains coffee, tea, or water. You taste it (runtime check).
- **Generic Cup**:
  - Cup → must contain coffee.
  - Cup → must contain tea.

  The barista (compiler) ensures correctness at the counter itself.

🔷 How this fits Java Generics

- **Box / Cup / Bag** → Generic Class.
- **T** = Type placeholder (like "Rice", "Apple", "Coffee").
- **Compiler** = Shopkeeper / Barista checking types.
- **Runtime errors avoided** because type safety is ensured upfront.

Where there is need of generics ?

When we write frameworks there generic is needed for type safty

Framework is a reusable code

Framework ⇨ library or package → Have some Public classes or Methods

This can be reused in the code → Other programmer use

Frameworks are created by community for social use, for Example React, Angular, Node.js

↓

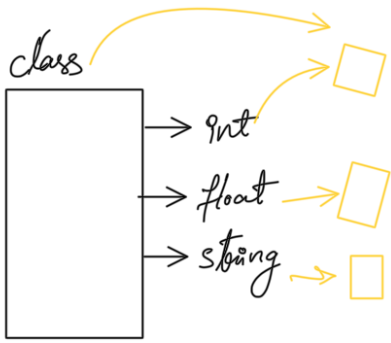
programmers around the globe will contribute to Build this frameworks

So to write this framework's we Need generics

Example

Write code for adding 2 inputs

```
10 + 20 → 30 int
10.5 + 11.5 → 21.5 float
a + b → ab str
```



Write a function/class of code which work for

integer, float, string concatenation.

So we define function in a generic way

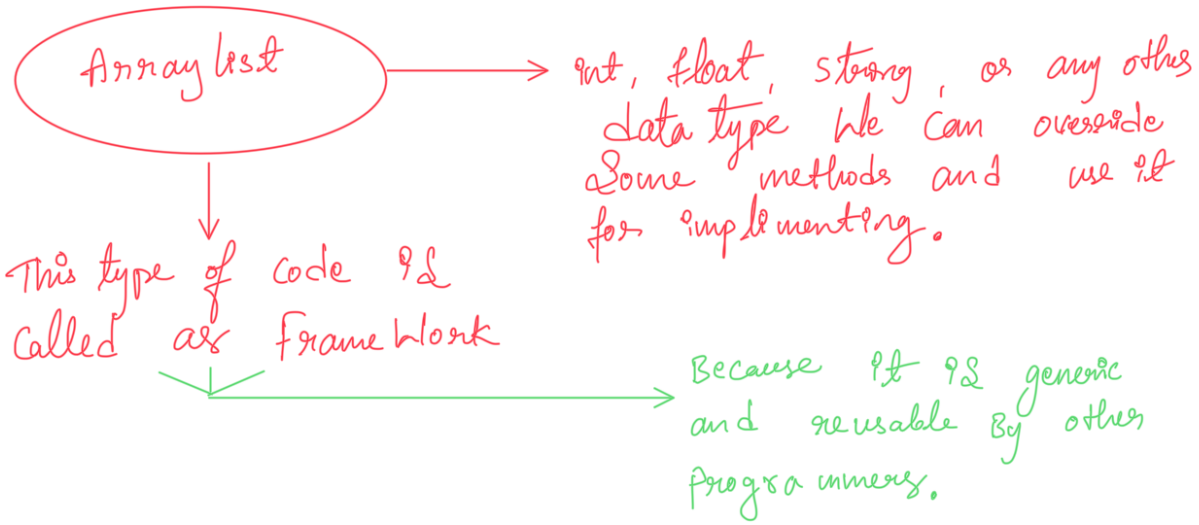
```
T   add (Ta , Tb )
{
        ____________        //defination
        ____________
        ____________

    return result ;
}
```

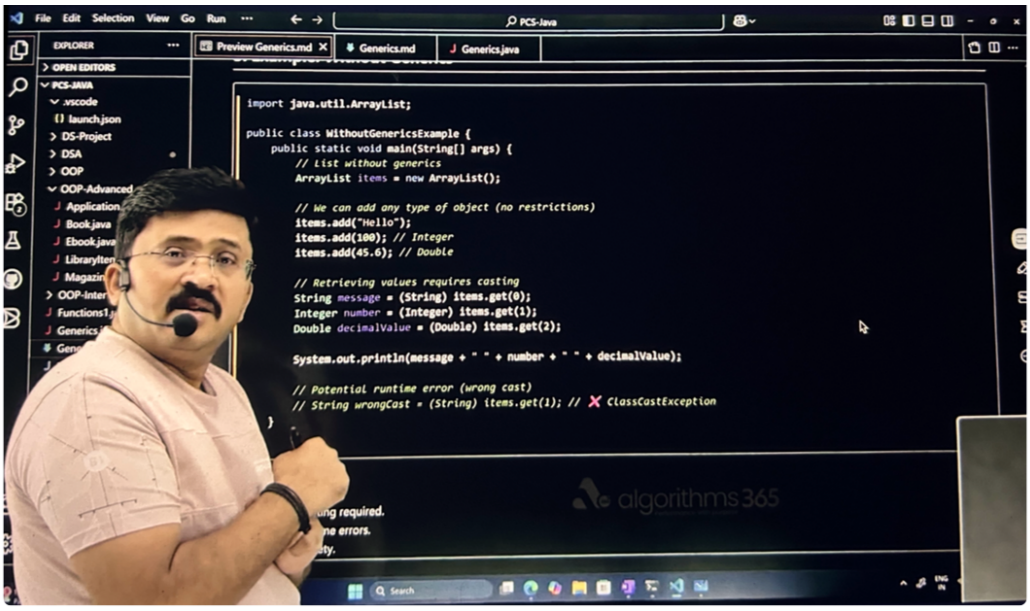
```
int ans = add (10, 10);
float ansf = add (10.5, 10.5);
String str = add ("Hi", "Hello");
```

Java is strongly typed so make it type safe we use generics

Arraylist ⟶ is a combination of Array & list



Without generics we have to type cast the object so to avoid type casting generics are used



Type Casting

java

```java
class GenericMethodExample {
    // Generic method with type parameter <T>
    public static <T> void printArray(T[] array) {
        for (T element : array) {
            System.out.print(element + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Integer[] intArr = {1, 2, 3};
        String[] strArr = {"A", "B", "C"};

        printArray(intArr);
        printArray(strArr);
    }
}
```

Example code

🔷 Generic Method Example

java

```java
class GenericMethodExample {
    // Generic method with type parameter <T>
    public static <T> void printArray(T[] array) {
        for (T element : array) {
            System.out.print(element + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Integer[] intArr = {1, 2, 3};
        String[] strArr = {"A", "B", "C"};

        printArray(intArr);
        printArray(strArr);
    }
}
```

Java Generics – Detailed Notes

1. Introduction

Generics in Java were introduced to **bring type safety and code reusability** to the language. Before generics, collections (like `ArrayList`, `HashMap`) stored objects as raw `Object` types, requiring explicit casting and risking `ClassCastException` at runtime.

When Introduced

- **Java 5 (2004)** introduced Generics as part of the Java Collections Framework improvements.
- Purpose: Enable developers to **write reusable, type-safe code** that works with multiple data types without rewriting logic.

---

2. Why Were Generics Introduced?

Problems Before Generics

- **Collections stored only `Object`** → Every time you retrieved a value, you had to cast it.
- **Prone to runtime errors** → Wrong type could be inserted unnoticed until a `ClassCastException` occurred.
- **Repetition of code** → Similar logic had to be written multiple times for each data type.

---

3. Example: Without Generics

```
import java.util.ArrayList;

public class WithoutGenericsExample {
    public static void main(String[] args) {
        // List without generics
        ArrayList items = new ArrayList();

        // We can add any type of object (no restrictions)
        items.add("Hello");
        items.add(100); // Integer
        items.add(45.6); // Double

        // Retrieving values requires casting
        String message = (String) items.get(0);
        Integer number = (Integer) items.get(1);
        Double decimalValue = (Double) items.get(2);

        System.out.println(message + " " + number + " " + decimalValue);

        // Potential runtime error (wrong cast)
        // String wrongCast = (String) items.get(1); // ❌ ClassCastException
    }
}
```

❌ Issues:

- Manual casting required.
- Risk of runtime errors.
- No type safety.

---

4. Example: With Generics

```
import java.util.ArrayList;

public class WithGenericsExample {
    public static void main(String[] args) {
        // List restricted to Strings only
        ArrayList<String> messages = new ArrayList<>();
        messages.add("Hello");
        messages.add("Generics");

        // No casting required when retrieving
        String firstMessage = messages.get(0);
        String secondMessage = messages.get(1);

        System.out.println(firstMessage + " " + secondMessage);

        // messages.add(100); // ❌ Compile-time error (type safety)
    }
}
```

✅ Advantages:

- Type safety enforced at compile time.
- No explicit casting required.
- Cleaner, reusable code.

---

5. Syntax of Generics

5.1 Declaring Generic Classes

```
class Box<T> {
    private T value;

    public void setValue(T value) {
        this.value = value;
    }

    public T getValue() {
        return value;
    }
}
```

Usage:

```
public class GenericClassExample {
    public static void main(String[] args) {
        Box<String> stringBox = new Box<>();
        stringBox.setValue("Hello Generics");
        System.out.println("String Value: " + stringBox.getValue());

        Box<Integer> integerBox = new Box<>();
        integerBox.setValue(123);
        System.out.println("Integer Value: " + integerBox.getValue());
    }
}
```

5.2 Declaring Generic Methods

```
public class GenericMethodExample {

    // Generic method to print arrays
    public static <T> void printArray(T[] array) {
        for (T element : array) {
            System.out.print(element + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Integer[] intArray = {10, 20, 30};
        String[] strArray = {"Java", "Generics", "Example"};

        printArray(intArray); // Works with Integer
        printArray(strArray); // Works with String
    }
}
```

5.3 Generics with Multiple Parameters

```
class KeyValuePair<K, V> {
    private K key;
    private V value;

    public KeyValuePair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public void printPair() {
        System.out.println("Key: " + key + " | Value: " + value);
    }
}

public class MultipleGenericsExample {
    public static void main(String[] args) {
        KeyValuePair<Integer, String> student = new KeyValuePair<>(101, "Alice");
        student.printPair();

        KeyValuePair<String, Double> product = new KeyValuePair<>("Laptop", 59999.99);
        product.printPair();
    }
}
```

---

6. Pros and Cons of Generics

✅ Pros

1. **Type Safety** – Prevents inserting wrong types at compile time.
2. **No Casting Needed** – Cleaner code.
3. **Reusability** – Same class/method works for multiple types.
4. **Readability** – Code is self-documenting about types used.
5. **Stronger Compile-Time Checking** – Errors caught early.

❌ Cons

1. **Cannot use primitive types directly** (`int`, `double` → need wrapper classes like `Integer`, `Double`).
2. **Type Erasure** – At runtime, type information is erased, limiting reflection capabilities.
3. **Array Creation with Generics not allowed** – Must use `List<T>` instead of `T[]`.

---

7. Generic Exercises

Exercise 1: Generic Addition Method

```
public class GenericAddition {
    // Generic method to add two numbers
    public static <T extends Number> double addNumbers(T firstNumber, T secondNumber) {
        return firstNumber.doubleValue() + secondNumber.doubleValue();
    }

    public static void main(String[] args) {
        System.out.println("Sum of Integers: " + addNumbers(10, 20));
        System.out.println("Sum of Doubles: " + addNumbers(10.5, 20.7));
    }
}
```

Exercise 2: Generic Concatenation Method

```
public class GenericConcatenation {
    // Generic method to concatenate two objects
    public static <T> String concatenateValues(T firstValue, T secondValue) {
        return firstValue.toString() + " " + secondValue.toString();
    }

    public static void main(String[] args) {
        System.out.println(concatenateValues("Hello", "World"));
        System.out.println(concatenateValues(100, 200));
        System.out.println(concatenateValues(45.6, 78.9));
    }
}
```

Exercise 3: Generic Printer Utility

```
public class GenericPrinter {
    // Generic method to print any type of array
    public static <T> void printElements(T[] elements) {
        for (T element : elements) {
            System.out.println(element);
        }
    }

    public static void main(String[] args) {
        String[] names = {"Alice", "Bob", "Charlie"};
        Integer[] numbers = {1, 2, 3, 4};

        System.out.println("Names:");
        printElements(names);

        System.out.println("Numbers:");
        printElements(numbers);
    }
}
```

---

8. Conclusion

Generics in Java:

- Were introduced in **Java 5**.
- Solve the problem of **type safety** and **code repetition**.
- Enable **reusable, type-safe, cleaner code**.
- Are widely used in Java Collections (`List<T>`, `Map<K, V>`, `Set<T>`).

 By mastering generics, developers can write **robust, scalable, and maintainable code**.